

Agentic Fund Platform - Complete Architecture Guide

1. Final Monolithic Architecture (Microservice-Ready)

```

agentic-fund-platform/
|
|   └── backend/
|   |   └── app/
|   |   |   ├── __init__.py
|   |   |   ├── main.py           # FastAPI app entry point
|   |   |   ├── config.py        # ServiceConfig with deployment modes
|   |   |   └── startup.py        # Service registration and initialization
|   |   |
|   |   |   └── core/             # Shared core functionality
|   |   |   |   ├── __init__.py
|   |   |   |   ├── interfaces/   # Service interfaces/protocols
|   |   |   |   |   ├── __init__.py
|   |   |   |   |   ├── services.py # Protocol definitions
|   |   |   |   |   └── repositories.py # Repository interfaces
|   |   |   |
|   |   |   |   └── registry/      # Service registry
|   |   |   |   |   ├── __init__.py
|   |   |   |   |   ├── service_registry.py
|   |   |   |   |   └── container.py # DI container
|   |   |   |
|   |   |   |   └── state/         # State management
|   |   |   |   |   ├── __init__.py
|   |   |   |   |   ├── manager.py # StateManager
|   |   |   |   |   ├── stores.py   # Different state store implementations
|   |   |   |   |   └── workflow_state.py
|   |   |   |
|   |   |   |   └── monitoring/    # Metrics and monitoring
|   |   |   |   |   ├── __init__.py
|   |   |   |   |   ├── metrics.py # Prometheus metrics
|   |   |   |   |   ├── tracing.py  # Distributed tracing
|   |   |   |   |   └── health.py    # Health checks
|   |   |   |
|   |   |   |   └── agents/        # Base agent functionality
|   |   |   |   |   ├── __init__.py
|   |   |   |   |   ├── base.py     # ConfigurableAgent base
|   |   |   |   |   └── factory.py   # AgentFactory
|   |   |   |
|   |   |   |   └── services/      # Business services (future microservices)
|   |   |   |   |   ├── __init__.py
|   |   |   |   |
|   |   |   |   |   └── market_analysis/ # Market Analysis Service
|   |   |   |   |   |   ├── __init__.py
|   |   |   |   |   |   ├── service.py  # MarketAnalysisService
|   |   |   |   |   |   ├── repository.py # MarketDataRepository
|   |   |   |   |   |   ├── models.py    # Domain models
|   |   |   |   |   |   ├── agents.py    # Service-specific agents
|   |   |   |   |   |   └── api.py        # Internal API endpoints
|   |   |   |   |
|   |   |   |   |   └── document_processing/ # Document Service
|   |   |   |   |   |   ├── __init__.py
|   |   |   |   |   |   ├── service.py   # DocumentProcessingService
|   |   |   |   |   |   ├── repository.py # DocumentRepository
|   |   |   |   |   |   ├── models.py
|   |   |   |   |   |   ├── agents.py
|   |   |   |   |   |   └── api.py
|   |   |   |   |
|   |   |   |   |   └── investment_committee/ # Committee Service
|   |   |   |   |   |   ├── __init__.py
|   |   |   |   |   |   ├── service.py    # InvestmentCommitteeService
|   |   |   |   |   |   ├── repository.py
|   |   |   |   |   |   ├── models.py
|   |   |   |   |   |   ├── agents/       # Committee member agents
|   |   |   |   |   |   |   ├── buffett.py
|   |   |   |   |   |   |   ├── cathie_wood.py
|   |   |   |   |   |   |   └── ray_dalio.py
|   |   |   |   |   |   └── api.py
|   |   |   |   |
|   |   |   |   |   └── risk_assessment/ # Risk Service
|   |   |   |   |   |   ├── __init__.py
|   |   |   |   |   |   ├── service.py
|   |   |   |   |   |   ├── repository.py
|   |   |   |   |   |   ├── models.py
|   |   |   |   |   |   ├── agents.py
|   |   |   |   |   |   └── api.py
|   |   |   |   |
|   |   |   |   |   └── execution/       # Execution Service

```

```

├── __init__.py
├── service.py
├── repository.py
├── models.py
├── api.py
├── agent_orchestration/ # Orchestration Service
│   ├── __init__.py
│   ├── service.py # WorkflowOrchestrationService
│   └── workflows/
│       ├── investment_analysis.py
│       ├── portfolio_rebalance.py
│       └── fund_screening.py
│   └── api.py
├── prompt_management/ # Prompt Service
│   ├── __init__.py
│   ├── service.py # PromptManagementService
│   ├── repository.py # PromptRepository
│   ├── validator.py # Response validation
│   └── api.py
├── api/
│   ├── __init__.py
│   ├── dependencies.py # FastAPI dependencies
│   ├── public/ # Public API routes
│   │   ├── __init__.py
│   │   ├── analysis.py # Public analysis endpoints
│   │   ├── portfolios.py
│   │   └── auth.py
│   ├── internal/ # Internal service APIs
│   │   ├── __init__.py
│   │   ├── market_analysis.py
│   │   ├── document_processing.py
│   │   ├── investment_committee.py
│   │   ├── risk_assessment.py
│   │   └── execution.py
│   ├── admin/ # Admin APIs
│   │   ├── __init__.py
│   │   ├── prompts.py
│   │   ├── models.py
│   │   └── monitoring.py
│   ├── websocket/
│   │   ├── __init__.py
│   │   └── handlers.py
│   ├── infrastructure/ # Infrastructure components
│   │   ├── __init__.py
│   │   ├── database/
│   │   │   ├── __init__.py
│   │   │   ├── client.py # Supabase client wrapper
│   │   │   └── migrations.py
│   │   ├── cache/
│   │   │   ├── __init__.py
│   │   │   └── redis_client.py
│   │   ├── llm/
│   │   │   ├── __init__.py
│   │   │   ├── providers.py # LLM provider factory
│   │   │   └── clients.py # Specific LLM clients
│   │   ├── messaging/
│   │   │   ├── __init__.py
│   │   │   └── event_bus.py # Internal event bus
│   ├── shared/ # Shared utilities
│   │   ├── __init__.py
│   │   ├── models/ # Shared data models
│   │   │   ├── __init__.py
│   │   │   ├── requests.py # Request models
│   │   │   ├── responses.py # Response models
│   │   │   └── domain.py # Domain models

```

```
| | | └── utils/
| | |   ├── __init__.py
| | |   ├── json_parser.py
| | |   ├── validators.py
| | |   └── logger.py
| | |
| | | └── proxies/ # Remote service proxies
| | |   ├── __init__.py
| | |   ├── base.py # BaseServiceProxy
| | |   ├── market_analysis.py # RemoteMarketAnalysisService
| | |   └── document_processing.py
| | |
| | |
| | └── tests/
| |   ├── unit/
| |   ├── services/
| |   │   ├── core/
| |   │   └── integration/
| |   └── e2e/
| |
| |
| | └── scripts/
| | └── deployments/
| | └── requirements/
| | └── .env.example
| |
| | └── frontend/
| |   ├── src/
| |   ├── app/
| |   ├── components/
| |   ├── lib/
| |   ├── types/
| |   └── styles/
| | └── public/
| | └── package.json
| |
| | └── supabase/
| |   ├── migrations/
| |   ├── functions/
| |   └── seed/
| |
| |
| | └── docker/
| | └── scripts/
| | └── docs/
| | └── README.md
```

2. Database Schema

Core User Management

```
sql

-- Using Supabase Auth for users (auth.users table is managed by Supabase)

-- User preferences and settings
CREATE TABLE user_preferences (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  risk_tolerance TEXT CHECK (risk_tolerance IN ('conservative', 'moderate', 'aggressive')),
  investment_horizon TEXT CHECK (investment_horizon IN ('short', 'medium', 'long')),
  preferred_sectors TEXT[],
  esg_preference TEXT CHECK (esg_preference IN ('none', 'basic', 'strict')),
  notification_settings JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

AI Configuration Tables

```
sql
```

```

-- AI model configurations
CREATE TABLE ai_models (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT UNIQUE NOT NULL,
  provider TEXT NOT NULL CHECK (provider IN ('openai', 'anthropic', 'google', 'cohere')),
  model_id TEXT NOT NULL, -- 'gpt-4', 'claude-3-opus', etc
  parameters JSONB DEFAULT '{}', -- temperature, max_tokens, etc
  capabilities JSONB DEFAULT '[]', -- ['web_search', 'vision', 'function_calling']
  cost_per_1k_tokens DECIMAL(10, 6),
  rate_limit INTEGER, -- requests per minute
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Prompt templates with response formats
CREATE TABLE prompt_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT UNIQUE NOT NULL,
  agent_type TEXT NOT NULL,
  description TEXT,

  -- Input configuration
  prompt_text TEXT NOT NULL,
  variables JSONB NOT NULL DEFAULT '[]', -- ["symbol", "market_context"]

  -- Response configuration
  response_format TEXT CHECK (response_format IN ('json', 'text', 'structured')),
  response_schema JSONB, -- JSON schema for validation
  response_instructions TEXT,

  -- Validation and examples
  validation_rules JSONB DEFAULT '{}',
  parser_config JSONB DEFAULT '{}',
  example_input JSONB,
  example_output JSONB,

  -- Versioning
  version INTEGER DEFAULT 1,
  active BOOLEAN DEFAULT true,
  created_by UUID REFERENCES auth.users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Agent configurations
CREATE TABLE agent_configs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_name TEXT UNIQUE NOT NULL,
  agent_type TEXT NOT NULL,
  description TEXT,

  -- Model assignment
  model_id UUID REFERENCES ai_models(id),
  fallback_model_id UUID REFERENCES ai_models(id),

  -- Configuration
  config JSONB DEFAULT '{}', -- Agent-specific settings
  max_retries INTEGER DEFAULT 3,
  timeout_seconds INTEGER DEFAULT 300,

  -- Status
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Agent-prompt mappings
CREATE TABLE agent_prompts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agent_configs(id) ON DELETE CASCADE,
  prompt_id UUID REFERENCES prompt_templates(id) ON DELETE CASCADE,
  prompt_type TEXT NOT NULL, -- 'research', 'analysis', 'synthesis'
  priority INTEGER DEFAULT 1,
  UNIQUE(agent_id, prompt_type)
);

```

Workflow Configuration

```
sql

-- Workflow definitions for LangGraph
CREATE TABLE workflow_configs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT UNIQUE NOT NULL,
  description TEXT,
  workflow_type TEXT NOT NULL,

  -- LangGraph configuration
  graph_definition JSONB NOT NULL, -- Nodes, edges, conditions
  default_timeout INTEGER DEFAULT 600,
  max_iterations INTEGER DEFAULT 50,

  -- Status
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Workflow execution history
CREATE TABLE workflow_executions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id UUID REFERENCES workflow_configs(id),
  user_id UUID REFERENCES auth.users(id),

  -- Execution details
  input_data JSONB NOT NULL,
  state_history JSONB DEFAULT '[]',
  output_data JSONB,

  -- Status tracking
  status TEXT CHECK (status IN ('pending', 'running', 'completed', 'failed', 'timeout')),
  started_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ,
  error_message TEXT,

  -- Metrics
  total_tokens_used INTEGER,
  total_cost DECIMAL(10, 4),
  execution_time_ms INTEGER
);
```

Financial Data Tables

```
sql
```

```

-- Fund universe
CREATE TABLE funds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  symbol TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  fund_type TEXT, -- 'mutual_fund', 'etf', 'index'
  category TEXT,
  expense_ratio DECIMAL(5, 4),
  inception_date DATE,
  fund_company TEXT,
  total_assets DECIMAL(20, 2),

  -- Cached metrics
  ytd_return DECIMAL(8, 4),
  one_year_return DECIMAL(8, 4),
  three_year_return DECIMAL(8, 4),
  five_year_return DECIMAL(8, 4),

  -- Metadata
  last_updated TIMESTAMPTZ DEFAULT NOW(),
  active BOOLEAN DEFAULT true
);

-- User portfolios
CREATE TABLE portfolios (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  description TEXT,

  -- Portfolio metrics
  total_value DECIMAL(20, 2),
  cash_balance DECIMAL(20, 2) DEFAULT 0,

  -- Status
  is_primary BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(user_id, name)
);

-- Portfolio holdings
CREATE TABLE portfolio_holdings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  portfolio_id UUID REFERENCES portfolios(id) ON DELETE CASCADE,
  fund_id UUID REFERENCES funds(id),

  -- Position details
  quantity DECIMAL(20, 8) NOT NULL,
  average_cost DECIMAL(20, 4),
  current_value DECIMAL(20, 2),

  -- Tracking
  first_purchased TIMESTAMPTZ,
  last_updated TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(portfolio_id, fund_id)
);

-- Transaction history
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  portfolio_id UUID REFERENCES portfolios(id) ON DELETE CASCADE,
  fund_id UUID REFERENCES funds(id),

  -- Transaction details
  transaction_type TEXT CHECK (transaction_type IN ('buy', 'sell', 'dividend', 'fee')),
  quantity DECIMAL(20, 8),
  price DECIMAL(20, 4),
  total_amount DECIMAL(20, 2),
  fees DECIMAL(10, 2) DEFAULT 0,

  -- Agent decision reference
  agent_decision_id UUID REFERENCES agent_decisions(id),

  -- Timestamps

```

```
executed_at TIMESTAMPTZ DEFAULT NOW(),  
settlement_date DATE  
);
```

Agent Decision & Learning Tables

sql


```

-- Agent decisions and analysis
CREATE TABLE agent_decisions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  portfolio_id UUID REFERENCES portfolios(id),

  -- Decision context
  decision_type TEXT NOT NULL, -- 'buy', 'sell', 'hold', 'rebalance'
  query TEXT,
  fund_symbol TEXT,

  -- Analysis results
  market_analysis JSONB,
  committee_analyses JSONB, -- Array of committee member analyses
  risk_assessment JSONB,
  final_recommendation JSONB,

  -- Execution plan
  recommended_action TEXT,
  recommended_amount DECIMAL(20, 2),
  confidence_score DECIMAL(5, 2) CHECK (confidence_score >= 0 AND confidence_score <= 100),

  -- Research trail
  research_sources JSONB, -- All sources consulted
  web_searches JSONB, -- Search queries and results
  documents_analyzed JSONB, -- Documents processed

  -- Timestamps
  created_at TIMESTAMPTZ DEFAULT NOW(),
  executed_at TIMESTAMPTZ,

  -- Outcome tracking
  outcome_measured_at TIMESTAMPTZ,
  outcome_metrics JSONB
);

-- Agent memory and learning
CREATE TABLE agent_memories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_name TEXT NOT NULL,

  -- Memory content
  context JSONB NOT NULL,
  decision JSONB NOT NULL,
  outcome JSONB,
  lessons_learned JSONB,

  -- Embeddings for similarity search
  context_embedding vector(1536),

  -- Metadata
  importance_score DECIMAL(3, 2) DEFAULT 0.5,
  access_count INTEGER DEFAULT 0,
  last_accessed TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Performance tracking
CREATE TABLE agent_performance (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_name TEXT NOT NULL,
  period_start DATE NOT NULL,
  period_end DATE NOT NULL,

  -- Metrics
  total_decisions INTEGER DEFAULT 0,
  successful_decisions INTEGER DEFAULT 0,
  average_confidence DECIMAL(5, 2),
  average_return DECIMAL(8, 4),

  -- Detailed metrics
  metrics_by_decision_type JSONB,
  metrics_by_market_regime JSONB,

  UNIQUE(agent_name, period_start, period_end)
);

```

Document Intelligence Tables

```
sql

-- Document storage and analysis
CREATE TABLE document_intelligence (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),

  -- Document info
  file_path TEXT NOT NULL,
  file_name TEXT NOT NULL,
  document_type TEXT, -- 'earnings_transcript', 'sec_filing', 'research_report'
  source TEXT, -- 'upload', 'mcp', 'web_scrape'

  -- Analysis results
  raw_content TEXT,
  structured_analysis JSONB,
  key_insights JSONB,
  entities_extracted JSONB, -- Companies, people, dates mentioned

  -- Embeddings for search
  content_embedding vector(1536),

  -- Metadata
  document_date DATE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ -- For temporary documents
);

-- MCP server registry
CREATE TABLE mcp_servers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT UNIQUE NOT NULL,
  endpoint TEXT NOT NULL,
  server_type TEXT,
  capabilities JSONB,
  auth_config JSONB, -- Encrypted auth details

  -- Health tracking
  last_health_check TIMESTAMPTZ,
  is_healthy BOOLEAN DEFAULT true,
  error_count INTEGER DEFAULT 0,

  -- Status
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

System & Monitoring Tables

sql

```

-- API usage tracking
CREATE TABLE api_usage (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),

  -- Request details
  endpoint TEXT NOT NULL,
  method TEXT NOT NULL,
  request_body JSONB,
  response_status INTEGER,

  -- Performance
  response_time_ms INTEGER,
  tokens_used INTEGER,

  -- Timestamps
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- System events and alerts
CREATE TABLE system_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type TEXT NOT NULL,
  severity TEXT CHECK (severity IN ('info', 'warning', 'error', 'critical')),

  -- Event details
  message TEXT NOT NULL,
  details JSONB,
  service_name TEXT,

  -- Resolution
  resolved BOOLEAN DEFAULT false,
  resolved_at TIMESTAMPTZ,
  resolved_by UUID REFERENCES auth.users(id),

  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

Indexes and Performance

```

sql

-- User and portfolio indexes
CREATE INDEX idx_portfolios_user_id ON portfolios(user_id);
CREATE INDEX idx_holdings_portfolio_id ON portfolio_holdings(portfolio_id);
CREATE INDEX idx_transactions_portfolio_id ON transactions(portfolio_id);
CREATE INDEX idx_transactions_executed_at ON transactions(executed_at DESC);

-- Agent decision indexes
CREATE INDEX idx_decisions_user_id ON agent_decisions(user_id);
CREATE INDEX idx_decisions_created_at ON agent_decisions(created_at DESC);
CREATE INDEX idx_decisions_fund_symbol ON agent_decisions(fund_symbol);

-- Vector similarity search indexes
CREATE INDEX idx_memories_embedding ON agent_memories
USING ivfflat (context_embedding vector_cosine_ops) WITH (lists = 100);

CREATE INDEX idx_documents_embedding ON document_intelligence
USING ivfflat (content_embedding vector_cosine_ops) WITH (lists = 100);

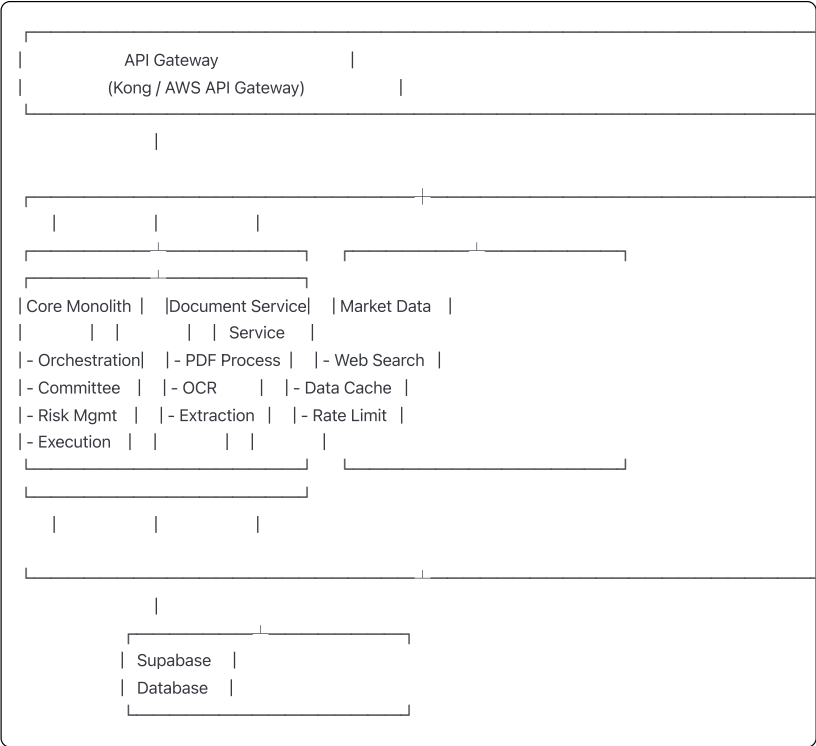
-- Full text search
CREATE INDEX idx_documents_content ON document_intelligence
USING gin(to_tsvector('english', raw_content));

-- Workflow execution indexes
CREATE INDEX idx_workflow_exec_user_id ON workflow_executions(user_id);
CREATE INDEX idx_workflow_exec_status ON workflow_executions(status);
CREATE INDEX idx_workflow_exec_started_at ON workflow_executions(started_at DESC);

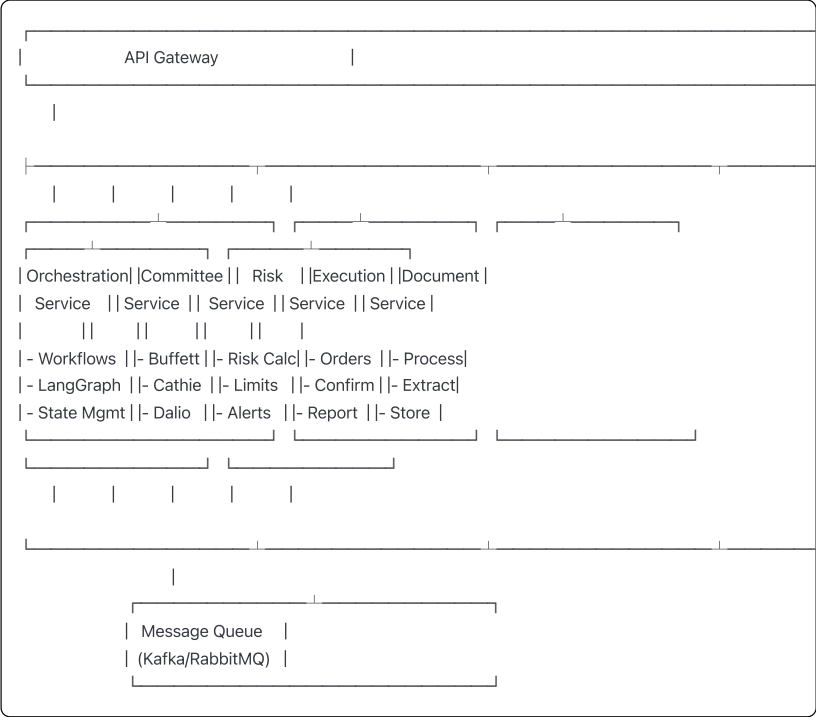
```

3. Future Microservices Architecture

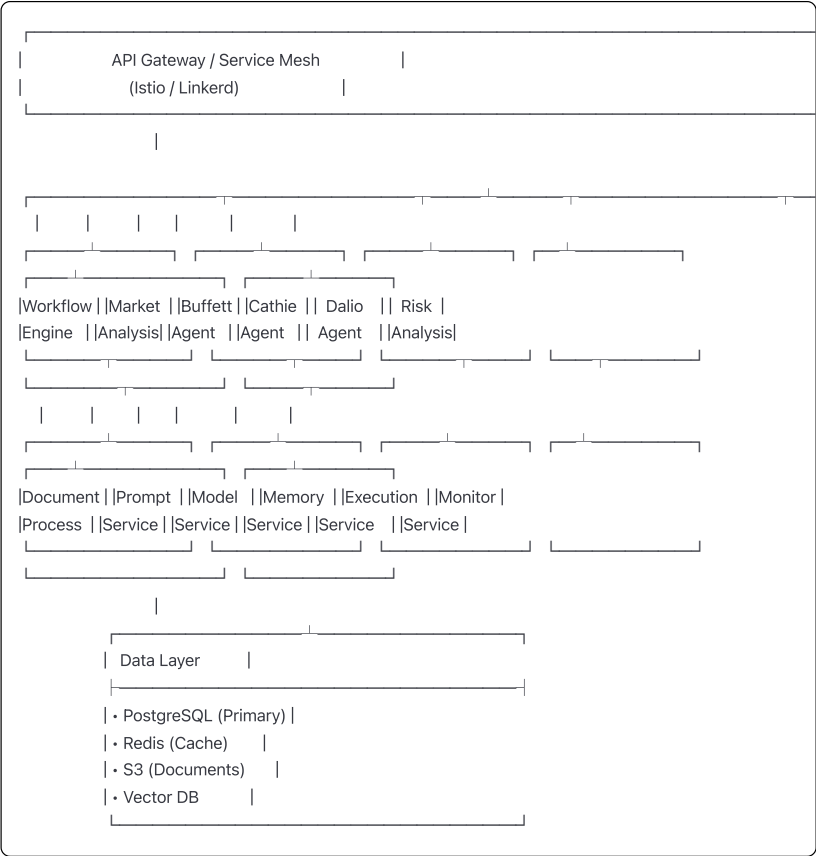
Phase 1: Initial Separation (6-12 months)



Phase 2: Core Service Separation (12-18 months)



Phase 3: Full Microservices (18+ months)



Service Communication Patterns

- 1. Synchronous (REST/gRPC):
 - User-facing APIs
 - Real-time queries
 - Critical path operations
- 2. Asynchronous (Message Queue):
 - Agent analysis workflows
 - Document processing
 - Batch operations
 - Event notifications
- 3. Event Streaming (Kafka):
 - Market data updates
 - Portfolio changes
 - System events
 - Audit logs

Deployment Strategy

Development → Staging → Production

Tools:

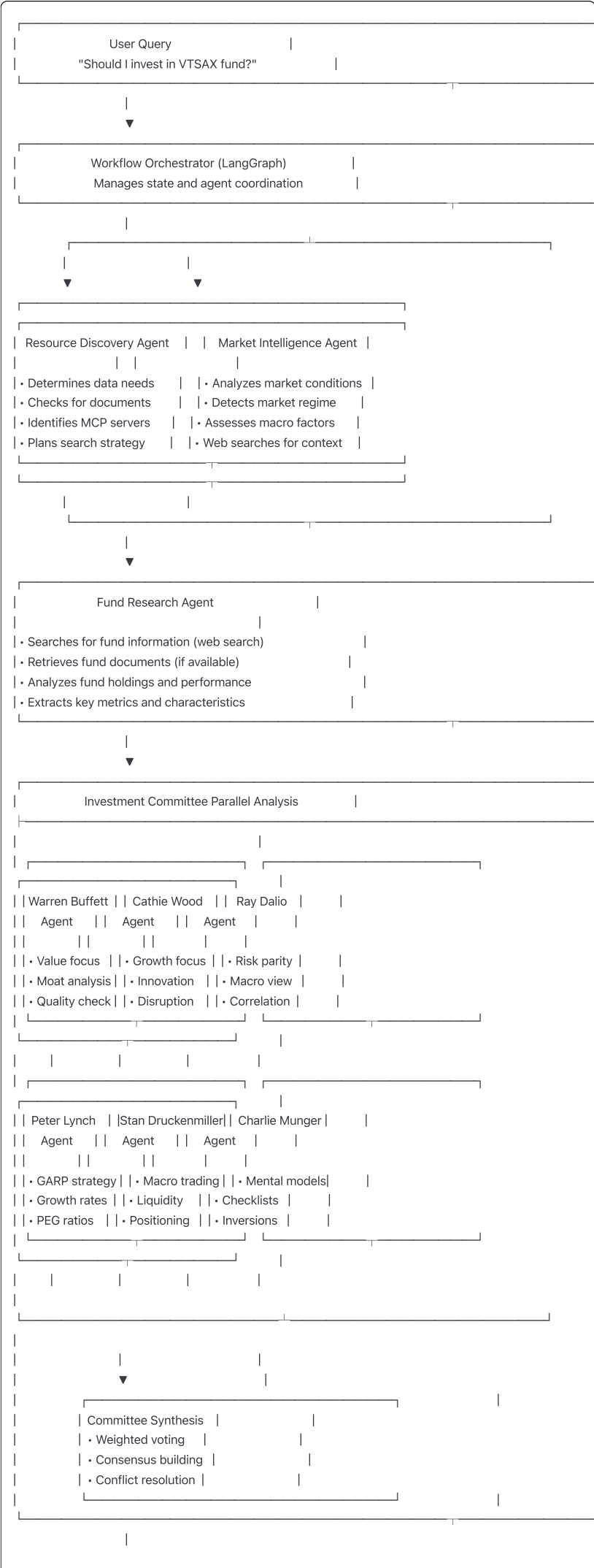
- Kubernetes for orchestration
- Helm for package management
- ArgoCD for GitOps
- Istio for service mesh
- Prometheus + Grafana for monitoring
- ELK stack for logging
- Jaeger for distributed tracing

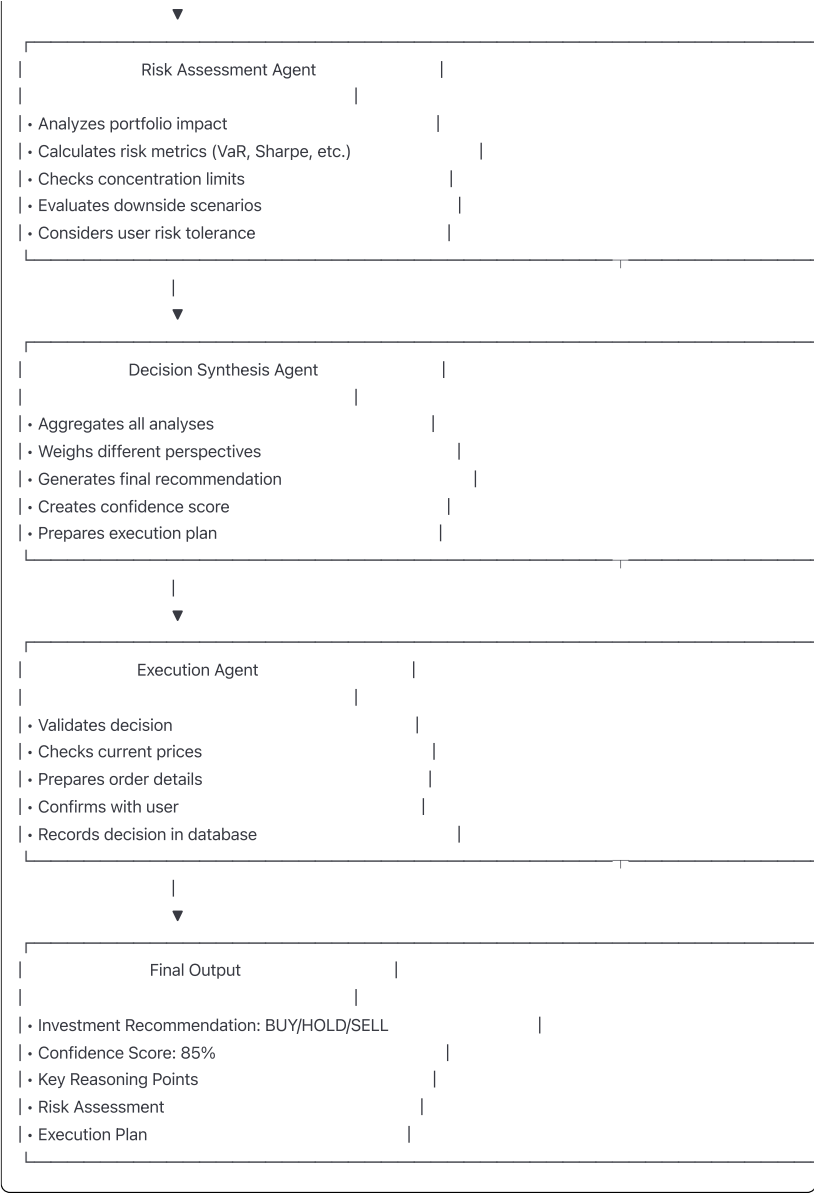
Migration Priorities

- 1. Document Processing Service (High CPU/Memory usage)
- 2. Market Data Service (External API rate limits)
- 3. Individual Agent Services (Independent scaling needs)
- 4. Risk Analysis Service (Computational intensity)
- 5. Execution Service (Reliability requirements)

This architecture provides a clear path from monolith to microservices while maintaining flexibility and operational efficiency at each stage.

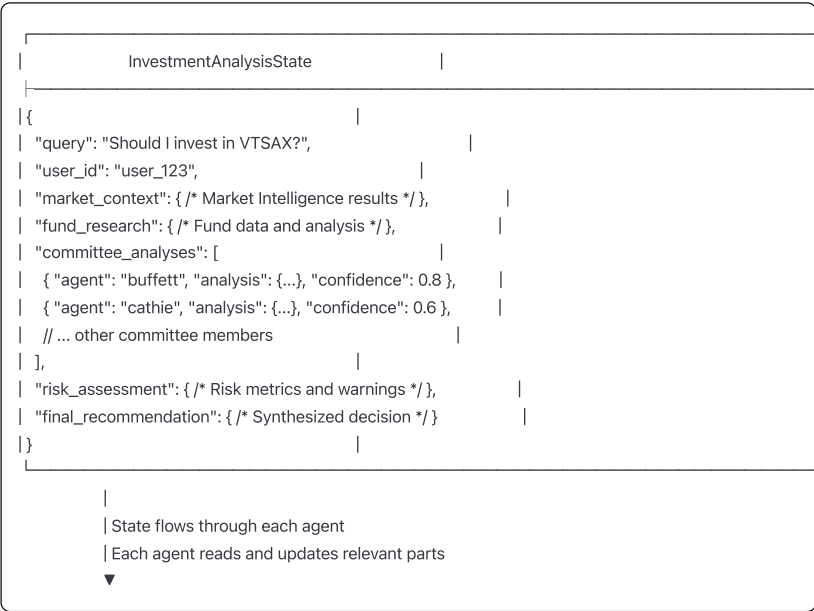
4. Agent Workflow and Collaboration



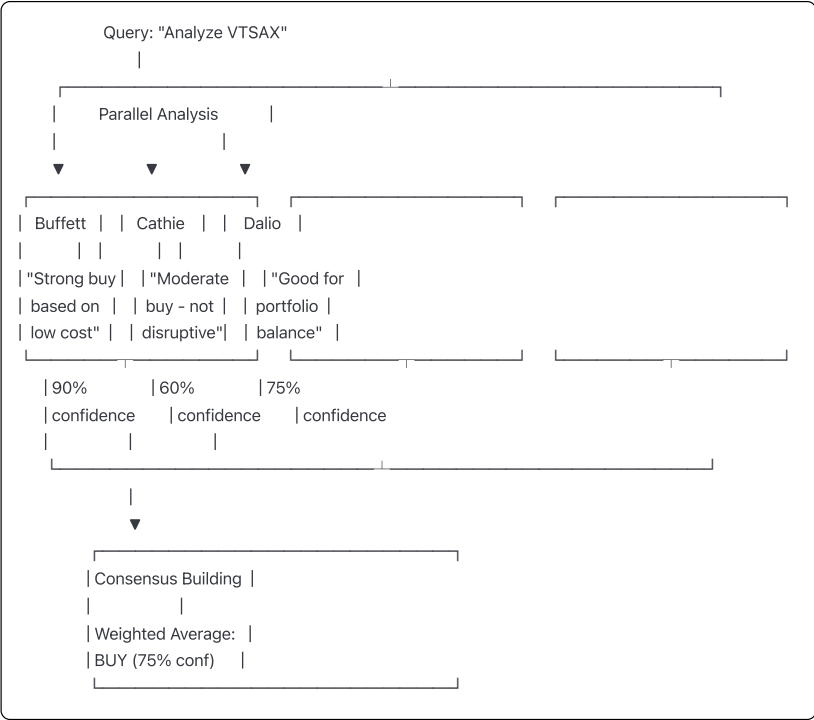


Agent Communication Patterns

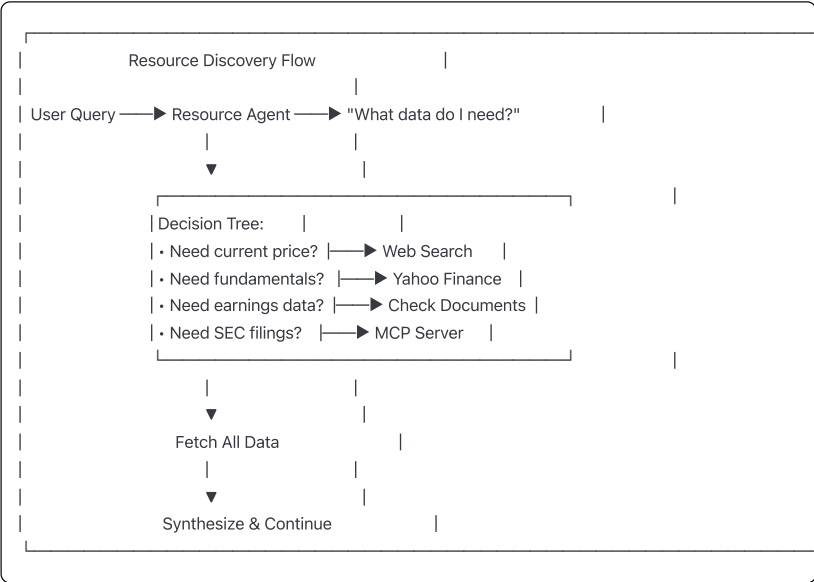
1. State Management Flow



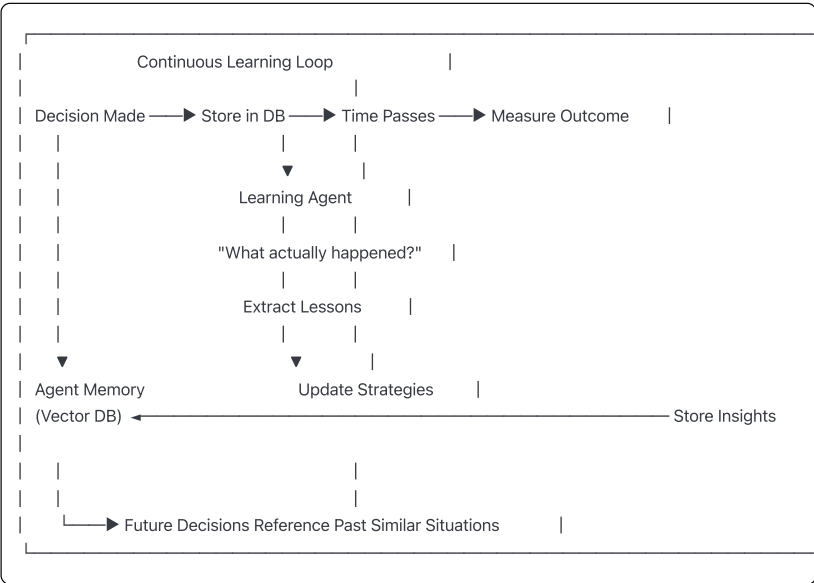
2. Committee Collaboration Process



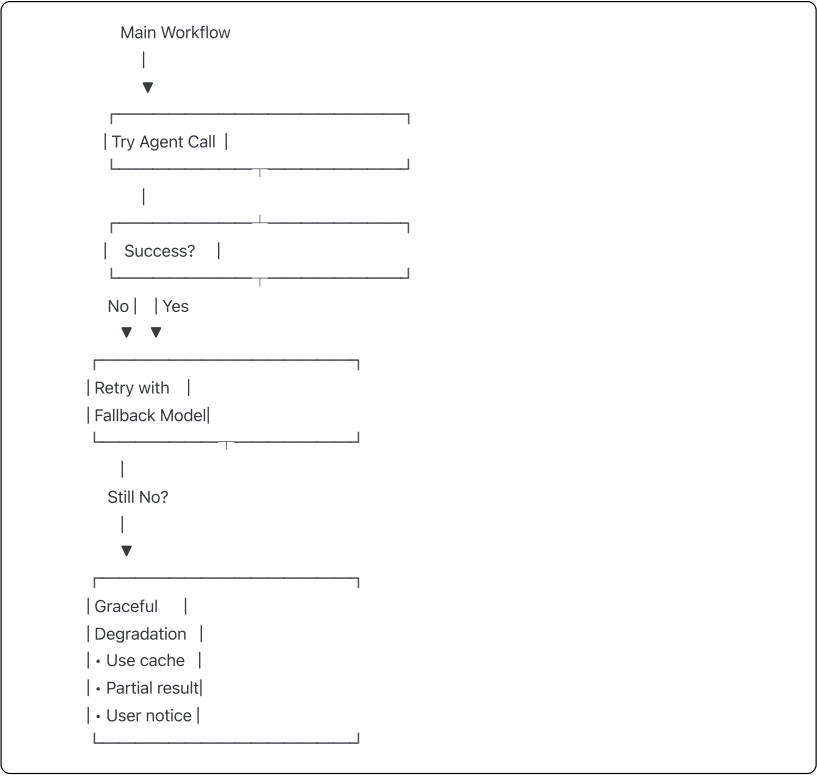
3. Dynamic Resource Discovery



4. Learning and Memory Flow



Error Handling and Fallback Workflow



This workflow architecture ensures robust, collaborative decision-making with multiple perspectives, proper error handling, and continuous learning capabilities.

LangGraph State Machine Implementation

```
python
```

```

# Simplified workflow definition showing agent coordination
from langgraph.graph import StateGraph, END

# State definition
class InvestmentAnalysisState(TypedDict):
    query: str
    user_id: str
    market_context: dict
    fund_research: dict
    committee_analyses: List[dict]
    risk_assessment: dict
    final_recommendation: dict
    confidence_score: float
    error_state: Optional[dict]

# Build the workflow graph
workflow = StateGraph(InvestmentAnalysisState)

# Add nodes (each node is an agent)
workflow.add_node("resource_discovery", resource_discovery_agent)
workflow.add_node("market_analysis", market_intelligence_agent)
workflow.add_node("fund_research", fund_research_agent)
workflow.add_node("committee", investment_committee_parallel)
workflow.add_node("risk_assessment", risk_assessment_agent)
workflow.add_node("synthesis", decision_synthesis_agent)
workflow.add_node("execution", execution_agent)

# Define edges (workflow flow)
workflow.add_edge("resource_discovery", "market_analysis")
workflow.add_edge("market_analysis", "fund_research")
workflow.add_edge("fund_research", "committee")
workflow.add_edge("committee", "risk_assessment")
workflow.add_edge("risk_assessment", "synthesis")

# Conditional edge based on confidence
workflow.add_conditional_edges(
    "synthesis",
    lambda state: "execute" if state["confidence_score"] > 0.7 else "need_human",
    {
        "execute": "execution",
        "need_human": END
    }
)

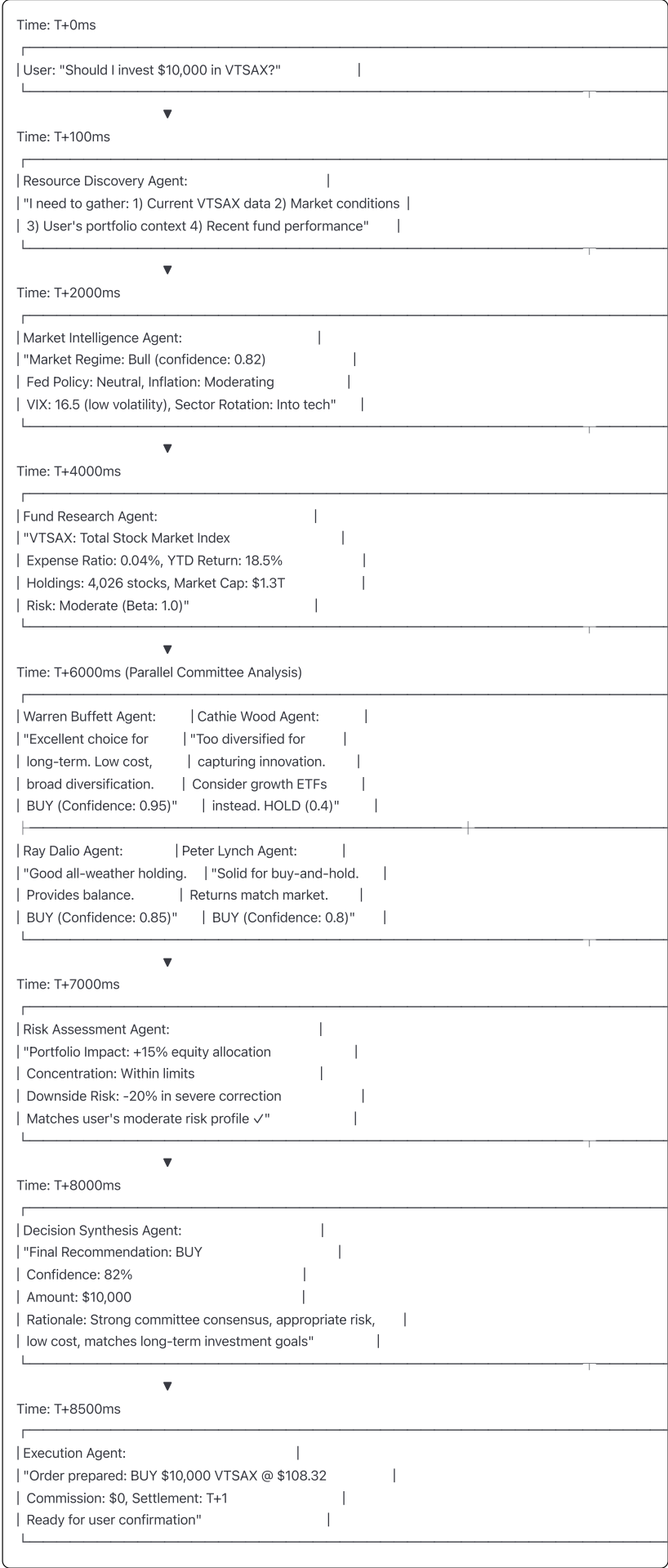
workflow.add_edge("execution", END)

# Set entry point
workflow.set_entry_point("resource_discovery")

# Compile the workflow
app = workflow.compile()

```

Real-time Agent Collaboration Example



This complete workflow architecture shows how agents collaborate through LangGraph's state management system to provide comprehensive investment analysis and recommendations.

Agent Information Sharing and Decision Mechanics

1. Information Flow Between Agents



python

```
# How agents communicate through the state
class AgentProtocol:
    async def process(self, state: InvestmentAnalysisState) -> InvestmentAnalysisState:
        # 1. Read relevant context from state
        context = self.extract_context(state)

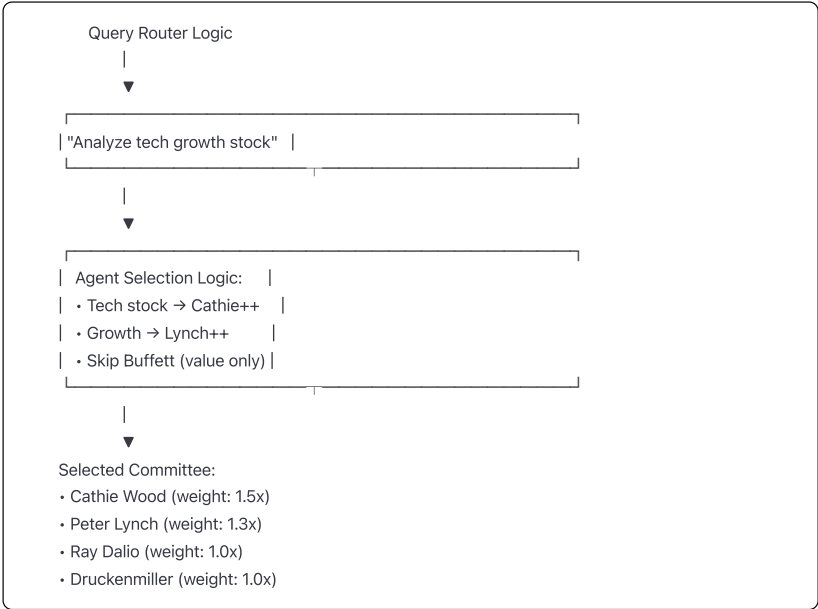
        # 2. Perform analysis using own expertise
        analysis = await self.analyze(context)

        # 3. Update state with findings
        state = self.update_state(state, analysis)

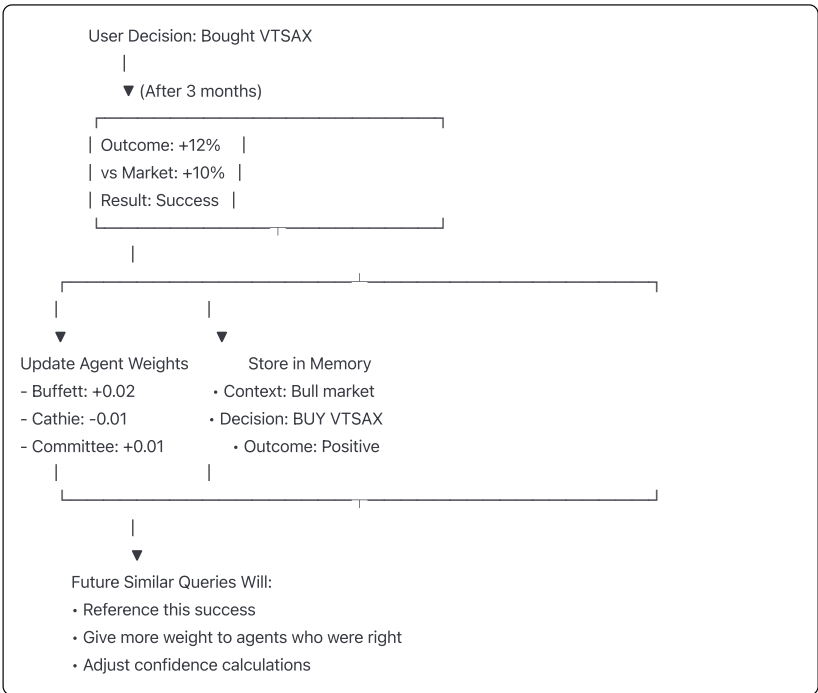
        # 4. Log decision rationale
        state['reasoning_chain'].append({
            'agent': self.name,
            'timestamp': datetime.now(),
            'inputs_used': context.keys(),
            'conclusion': analysis.conclusion,
            'confidence': analysis.confidence
        })

    return state
```

4. Dynamic Agent Selection Based on Query



5. Feedback Loop Integration



This detailed workflow architecture demonstrates how agents collaborate, share information, make consensus decisions, and continuously improve through feedback loops.